



Fragments

& Jetpack Compose

Corso di Laboratorio di Programmazione per Sistemi Mobile e Tablet
2025/2026

Cos'è un Fragment

Un Fragment è una porzione riutilizzabile di interfaccia utente con il proprio ciclo di vita, ospitata all'interno di un'Activity.

- Ha un proprio ciclo di vita, legato a quello dell'Activity host.
- Può essere aggiunto, rimosso o sostituito a runtime (transazioni).
- Permette di costruire interfacce multi-pannello (tablet, foldable).
- È il componente standard per la navigazione con Navigation Component.
- Introdotto in Android 3.0 (API 11), ora parte di AndroidX.
- Può ospitare contenuto Compose tramite ComposeView.



Fragment vs Activity

Un'Activity è un punto d'ingresso dell'app (con un proprio task/back stack). Un Fragment vive dentro un'Activity e rappresenta una porzione della schermata.

Quando usare i Fragment

I Fragment sono utili in molti scenari, ma non sempre sono la scelta migliore.



Layout multi-pannello

Tablet: lista a sinistra + dettaglio a destra nella stessa Activity.



Navigazione in-app

Navigation Component usa Fragment come destinazioni del grafo.



UI riutilizzabile

Un Fragment di login può essere usato in più Activity.



Interop con Compose

ComposeView permette di inserire UI Compose dentro un Fragment esistente.



In nuovi progetti 100% Compose, i Fragment non servono più: ogni schermata è un composable nel NavHost.

Ciclo di vita del Fragment

Il Fragment ha callback propri che si intrecciano con quelli dell'Activity host.

onAttach()

legato all'Activity host

onCreate()

inizializzazione (NO view)

onCreateView()

crea la view hierarchy

onViewCreated()

configura le view (binding)

onStart()

visibile all'utente

onResume()

interattivo (in primo piano)

onPause()

non più in primo piano

onStop()

non più visibile

onDestroyView()

view distrutta (rilascia binding!)

onDestroy()

Fragment distrutto

onDetach()

staccato dall'Activity



La view ha un ciclo di vita diverso dal Fragment! Rilascia sempre il binding in `onDestroyView()`.

Creare un Fragment

Un Fragment si crea estendendo la classe `Fragment` e sovrascrivendo `onCreateView` o `onViewCreated`.

```
class HomeFragment : Fragment(R.layout.fragment_home) {  
  
    // ViewBinding sicuro con il ciclo di vita  
    private var _binding: FragmentHomeBinding? = null  
    private val binding get() = _binding!!  
  
    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {  
        super.onViewCreated(view, savedInstanceState)  
        _binding = FragmentHomeBinding.bind(view)  
  
        binding.titleText.text = "Benvenuto"  
        binding.actionBtn.setOnClickListener {  
            // gestisci il click  
        }  
    }  
  
    override fun onDestroyView() {  
        super.onDestroyView()  
        _binding = null // IMPORTANTE: evita memory leak  
    }  
}
```

Kotlin

FragmentManager: add, replace, remove

Le transazioni permettono di modificare i Fragment visibili in un container a runtime.

```
// Ottenere il FragmentManager
val fm = supportFragmentManager

// REPLACE: sostituisce il contenuto
fm.commit {
    replace(R.id.container, HomeFragment())
    addToBackStack("home")
}

// ADD: aggiunge sopra quello esistente
fm.commit {
    add(R.id.container, DetailFragment())
    addToBackStack("detail")
}

// REMOVE: rimuove un fragment
fm.commit {
    remove(detailFragment)
}

// Tornare indietro nel back stack
fm.popBackStack()
```

Kotlin

Concetti chiave

- `replace()` rimuove il vecchio e aggiunge il nuovo.
- `add()` sovrappone senza rimuovere.
- `addToBackStack()` permette il tasto Indietro.
- `commit()` è asincrono. `commitNow()` è sincrono.
- `commit { }` è l'estensione Kotlin di KTX.

Il container nell'Activity

L'Activity host contiene un `FrameLayout` (o `FragmentContainerView`) dove i `Fragment` vengono inseriti.

activity_main.xml

```
<androidx.fragment.app.FragmentContainerView
    android:id="@+id/container"
    android:name=
        "com.example.app.HomeFragment"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    app:defaultNavHost="true"
    app:navGraph="@navigation/nav_graph"/>
```

XML

MainActivity.kt

```
class MainActivity : AppCompatActivity(
    R.layout.activity_main
) {
    // Con Navigation Component
    // il FragmentContainerView gestisce
    // tutto automaticamente.
    // Non serve commit() manuale.
}
```

Kotlin



FragmentContainerView vs FrameLayout

`FragmentContainerView` è il container raccomandato: gestisce le animazioni `fragment`, supporta `defaultNavHost` e l'attributo `android:name` per il `fragment` iniziale.

Comunicazione tra Fragment

I Fragment non dovrebbero comunicare direttamente tra loro. Esistono due pattern raccomandati.

1. ViewModel condiviso (shared)

```
// ViewModel condiviso tra i Fragment                                Kotlin
class SharedVM : ViewModel() {
    val selected = MutableLiveData<Item>()
}

// Nel ListFragment
class ListFragment : Fragment() {
    private val vm: SharedVM
        by activityViewModels()
    fun onItemClick(item: Item) {
        vm.selected.value = item
    }
}

// Nel DetailFragment
class DetailFragment : Fragment() {
    private val vm: SharedVM
        by activityViewModels()
}
```

2. Fragment Result API

```
// Nel Fragment che INVIA il risultato                                Kotlin
setFragmentResult(
    "requestKey",
    bundleOf("data" to "valore")
)

// Nel Fragment che RICEVE
setFragmentResultListener(
    "requestKey"
) { _, bundle ->
    val data = bundle.getString("data")
    binding.text.text = data
}

// Funziona tra parent-child e
// tra sibling via Activity scope.
```



MAI passare dati con il costruttore del Fragment. Usare sempre arguments Bundle o i pattern sopra.

Navigation Component con Fragment

Il Navigation Component gestisce le transazioni, il back stack, le animazioni e gli argomenti in modo dichiarativo.

```
<!-- res/navigation/nav_graph.xml -->
<navigation
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    app:startDestination="@id/homeFragment">

    <fragment
        android:id="@+id/homeFragment"
        android:name="com.example.HomeFragment">
        <action
            android:id="@+id/toDetail"
            app:destination="@id/detailFragment"/>
        </fragment>

    <fragment
        android:id="@+id/detailFragment"
        android:name="com.example.DetailFragment">
        <argument
            android:name="itemId"
            app:argType="integer"/>
        </fragment>
</navigation>
```

XML

Navigare dal codice

```
// Nel HomeFragment
findNavController().navigate(
    R.id.toDetail,
    bundleOf("itemId" to 42)
)
```

Kotlin

```
// Con SafeArgs (generato)
val action = HomeFragmentDirections
    .toDetail(itemId = 42)
findNavController()
    .navigate(action)
```

Kotlin

Compose dentro un Fragment

ComposeView permette di inserire contenuto Compose dentro un Fragment classico. È il primo passo della migrazione.

```
class HomeFragment : Fragment() {  
  
    override fun onCreateView(  
        inflater: LayoutInflater, container: ViewGroup?, savedInstanceState: Bundle?  
    ): View {  
        return ComposeView(requireContext()).apply {  
            setViewCompositionStrategy(  
                ViewCompositionStrategy.DisposeOnViewTreeLifecycleDestroyed  
            )  
            setContent {  
                MyAppTheme {  
                    HomeScreen(  
                        onNavigate = { id ->  
                            findNavController().navigate(R.id.toDetail, bundleOf("id" to id))  
                        }  
                    )  
                }  
            }  
        }  
    }  
}
```

Kotlin

ViewCompositionStrategy

La strategy controlla quando la composizione viene distrutta. Scegliere quella giusta evita memory leak.

RACCOMANDATO

DisposeOnViewTreeLifecycleDestroyed

Distrugge quando il LifecycleOwner della view tree viene distrutto. LA PIÙ COMUNE per Fragment.

DEFAULT

DisposeOnDetachedFromWindow

Distrugge quando la ComposeView viene staccata dalla finestra. Default, ma problematico nei Fragment con transazioni.

AVANZATO

DisposeOnLifecycleDestroyed(lifecycle)

Distrugge quando un lifecycle specifico viene distrutto. Per casi avanzati.

AndroidView: View dentro Compose

Se si ha un widget View che non ha ancora un equivalente Compose, si può ospitarlo con AndroidView.

```
@Composable
fun MapSection() {
    var mapView: MapView? = null

    AndroidView(
        factory = { context ->
            MapView(context).also { mv ->
                mv.onCreate(null)
                mv.getMapAsync { googleMap ->
                    // configura la mappa
                    googleMap.moveCamera(
                        CameraUpdateFactory.newLatLng(LatLng(45.0, 11.0))
                    )
                }
            }
            mapView = mv
        },
        update = { mv ->
            // chiamato a ogni ricomposizione
            // aggiorna la view se necessario
        },
        modifier = Modifier
            .fillMaxWidth()
            .height(300.dp)
    )
}
```

Kotlin

Compose sostituisce i Fragment

In un'app 100% Compose, ogni schermata è una `@Composable`. Non servono più `Fragment` né `FragmentManager`.

Concetto Fragment	Equivalente Compose
Fragment	<code>@Composable</code> fun
FragmentManager	<code>NavController</code>
FragmentTransaction	<code>navController.navigate()</code>
Back stack	<code>NavHost</code> gestisce automaticamente
<code>onCreateView()</code>	Il corpo della funzione <code>@Composable</code>
ViewBinding	Stato + composable (niente binding)
arguments Bundle	Parametri funzione o <code>NavArguments</code>
Fragment Result API	Callback / shared <code>ViewModel</code>

NavHost in Compose: esempio

NavHost sostituisce il NavGraph XML e FragmentContainerView. Tutto è Kotlin puro.

```
@Composable
fun AppNavigation() {
    val navController = rememberNavController()
    NavHost(navController, startDestination = "home") {
        composable("home") {
            HomeScreen(
                onItemClick = { id -> navController.navigate("detail/$id") }
            )
        }
        composable(
            "detail/{itemId}",
            arguments = listOf(navArgument("itemId") { type = NavType.IntType })
        ) { entry ->
            DetailScreen(
                itemId = entry.arguments?.getInt("itemId") ?: 0,
                onBackPressed = { navController.popBackStack() }
            )
        }
        composable("settings") {
            SettingsScreen()
        }
    }
}
```

Kotlin

Bottom Navigation in Compose

Scaffold + NavigationBar + NavHost: la struttura completa di un'app con tab bar.

```
sealed class Tab(val route: String, val label: String, val icon: ImageVector) {  
    object Home : Tab("home", "Home", Icons.Default.Home)  
    object Search : Tab("search", "Cerca", Icons.Default.Search)  
    object Profile : Tab("profile", "Profilo", Icons.Default.Person)  
}  
  
@Composable  
fun MainScreen() {  
    val navController = rememberNavController()  
    val tabs = listOf(Tab.Home, Tab.Search, Tab.Profile)  
    val current = navController.currentBackStackEntryAsState().value?.destination?.route  
  
    Scaffold(bottomBar = {  
        NavigationBar {  
            tabs.forEach { tab ->  
                NavigationBarItem(  
                    icon = { Icon(tab.icon, tab.label) },  
                    label = { Text(tab.label) },  
                    selected = current == tab.route,  
                    onClick = { navController.navigate(tab.route) {  
                        popUpTo("home") { saveState = true }; launchSingleTop = true; restoreState = true  
                    }}  
            )  
        }  
    }) { padding ->  
        NavHost(navController, "home", Modifier.padding(padding)) {  
            composable("home") { HomeScreen() }  
            composable("search") { SearchScreen() }  
            composable("profile") { ProfileScreen() }  
        }  
    }  
}
```

Kotlin

Strategia di migrazione graduale

Non è necessario riscrivere tutto in una volta. Google raccomanda una migrazione incrementale.

1

Nuove feature in Compose

Ogni nuova schermata è un composabile. I Fragment esistenti restano.

2

Compose dentro Fragment

Usa ComposeView per sostituire i layout XML dei Fragment esistenti uno alla volta.

3

Rimuovi i Fragment

Quando tutti i Fragment usano ComposeView, sostituisci il NavGraph XML con NavHost Compose.

4

100% Compose

Elimina le dipendenze Fragment. Una sola Activity, zero Fragment, tutto Compose.

ViewModel: funziona con entrambi

Il ViewModel è lo stesso sia che lo usi con Fragment che con Compose. È il ponte perfetto per la migrazione.

Nel Fragment (classico)

```
class HomeFragment : Fragment() {  
    private val vm: HomeVM  
        by viewModels()  
  
    override fun onCreateView(...) {  
        vm.items.observe(viewLifecycleOwner) {  
            adapter.submitList(it)  
        }  
    }  
}
```

Kotlin

In Compose

```
@Composable  
fun HomeScreen(  
    vm: HomeVM = viewModel()  
) {  
    val items by vm.items  
        .collectAsStateWithLifecycle()  
    LazyColumn {  
        items(items) { item ->  
            ItemRow(item)  
        }  
    }  
}
```

Kotlin

Stessa classe ViewModel, API diversa per osservare

Fragment: `observe(lifecycleOwner)`. Compose: `collectAsStateWithLifecycle()`. Il ViewModel non cambia.

Fragment vs Composable: confronto

Aspetto	Fragment	Composable
Ciclo di vita	11 callback complessi	Composizione automatica
Creazione UI	onCreateView + XML/Binding	Corpo della funzione @Composable
Stato	ViewModel + LiveData + observe	remember + mutableStateOf
Back stack	FragmentManager	NavController gestisce tutto
Comunicazione	Result API / shared VM	Parametri funzione / shared VM
Test UI	FragmentScenario	ComposeTestRule
Peso	Classe + layout XML + binding	Una funzione Kotlin
Raccomandazione	Progetti legacy	Nuovi progetti



I Fragment non sono deprecati, ma per nuovi progetti Google raccomanda Compose con Navigation Compose.

Best practice

DA FARE

- Rilascia `_binding` in `onDestroyView()`
- Usa `FragmentContainerView`, non `FrameLayout`
- Comunica via `ViewModel` condiviso
- Imposta `ViewCompositionStrategy` nei `Fragment`
- Nuove feature direttamente in `Compose`
- Naviga con `Navigation Component`
- `State hoisting` nei `composable`

DA EVITARE

- Passare dati tramite il costruttore
- Accedere a `binding` dopo `onDestroyView()`
- Transazioni manuali quando c'è `Navigation`
- `Fragment` annidati senza motivo
- Mescolare `FragmentManager` e `NavController`
- `DisposeOnDetachedFromWindow` nei `Fragment`
- `GlobalScope` dentro i `composable`

Errori comuni e soluzioni

NullPointerException su binding

Accesso a `_binding` dopo `onDestroyView()`. Soluzione: annulla `_binding = null`.

Composizione non distrutta

`ComposeView` senza la strategy corretta causa leak. Usa `DisposeOnViewTreeLifecycleDestroyed`.

Dati persi in rotazione

Stato nel `Fragment` anziché nel `ViewModel`. Sposta tutto nel VM.

IllegalStateException: Already added

Aggiungere lo stesso `Fragment` due volte. Usa `replace()` oppure controlla `isAdded`.

Back stack inconsistente

Mescolare `popBackStack()` e `navigate()`. Usa `Navigation Component` per gestirlo.

Fragment ricostruito senza argomenti

Dati passati nel costruttore vanno persi. Usa `arguments Bundle` o `SafeArgs`.

Cheat-sheet: Fragment

```
// 1. Creare un Fragment con layout
class MyFragment : Fragment(R.layout.fragment_my) { ... }

// 2. ViewBinding sicuro
private var _binding: FragmentMyBinding? = null
private val binding get() = _binding!!
override fun onDestroyView() { super.onDestroyView(); _binding = null }

// 3. Transazione
supportFragmentManager.commit {
    replace(R.id.container, MyFragment()); addToBackStack(null)
}

// 4. Navigazione
findNavController().navigate(R.id.action_to_detail, bundleOf("id" to 42))

// 5. ViewModel condiviso
private val sharedVM: SharedVM by activityViewModels()

// 6. Fragment Result API
setFragmentResult("key", bundleOf("data" to value))
setFragmentResultListener("key") { _, b -> val d = b.getString("data") }

// 7. Compose dentro Fragment
ComposeView(requireContext()).apply {
    setViewCompositionStrategy(DisposeOnViewTreeLifecycleDestroyed)
    setContent { MyAppTheme { MyScreen() } }
}
```

Kotlin

Cheat-sheet: Compose Navigation

```
// 1. NavHost
val navController = rememberNavController()
NavHost(navController, startDestination = "home") {
    composable("home") { HomeScreen(navController) }
    composable("detail/{id}", arguments = listOf(
        navArgument("id") { type = NavType.IntType }
    )) { entry -> DetailScreen(entry.arguments?.getInt("id") ?: 0) }
}

// 2. Navigare
navController.navigate("detail/42")

// 3. Tornare indietro
navController.popBackStack()

// 4. Bottom Navigation
NavigationBar {
    tabs.forEach { tab ->
        NavigationBarItem(
            selected = current == tab.route,
            onClick = { navController.navigate(tab.route) {
                popUpTo("home") { saveState = true }; launchSingleTop = true
            } },
            icon = { Icon(tab.icon, tab.label) }, label = { Text(tab.label) }
        )
    }
}
```

Kotlin